



Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)

КУРСОВОЙ ПРОЕКТ

«Музыкальный сервис по прослушиванию музыки»

Выполнил:

студент 3-го курса специальности

09.02.07 «Информационные системы и
программирование»

Горностаев Владислав Иванович

подпись: _____

Проверил:

преподаватель дисциплины,

преподаватель ЧУПО «ВШП»,

Честных Всеволод Викторович.

оценка: _____

подпись: _____

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	1
ВВЕДЕНИЕ.....	2
ОСНОВНАЯ ЧАСТЬ.....	7
Глава 1. Анализ предметной области музыкального сервиса.....	7
1.1 Характеристика музыкального сервиса как информационной системы.....	7
1.2 Ключевые бизнес-процессы, поддерживаемые базой данных.....	9
1.3 Требования к разрабатываемой базе данных.....	11
1.4 Выбор СУБД для реализации.....	12
Глава 2. Разработка и реализация базы данных музыкального сервиса.....	13
2.1 Логическое проектирование базы данных.....	13
2.2 Физическая реализация базы данных в MySQL.....	15
2.3 Ролевая модель и триггеры автоматизации.....	16
2.4 Создание представлений для упрощения доступа к данным.....	17
2.5 Реализация пользовательской функции.....	18
2.6 Разработка типовых запросов.....	18
2.7 Реализация транзакций и хранимых процедур.....	19
2.8 Наполнение базы данных тестовыми данными и проверка работоспособности. 20	
2.9 Выводы по второй главе.....	20
ЗАКЛЮЧЕНИЕ.....	21
СПИСОК ИСТОЧНИКОВ.....	25

ВВЕДЕНИЕ

Актуальность темы исследования. В эпоху цифровой трансформации

музыкальная индустрия переживает фундаментальные изменения. Традиционные способы распространения музыки через физические носители (винил, компакт-диски) и даже цифровые загрузки уступают место стриминговым сервисам, предоставляющим пользователям мгновенный доступ к миллионам музыкальных произведений. Согласно данным Международной федерации фонографической индустрии (IFPI), объем рынка потокового аудио в 2024 году превысил 20 миллиардов долларов США, а количество активных подписчиков музыкальных стриминговых сервисов достигло 600 миллионов человек. Такие платформы, как Spotify, Apple Music, Яндекс.Музыка, стали не просто инструментами для прослушивания музыки, а полноценными экосистемами, объединяющими пользователей, авторов, исполнителей и контент-мейкеров.

Современный сайт по прослушиванию музыки представляет собой сложную информационную систему, которая должна решать широкий спектр задач: хранение и каталогизацию музыкального контента, управление пользовательскими аккаунтами и профилями, ведение истории прослушиваний, формирование персональных рекомендаций, создание и редактирование плейлистов, реализацию системы избранного, а также разграничение прав доступа между обычными пользователями, авторами и администраторами. В основе любой такой системы лежит база данных, эффективность организации которой напрямую определяет скорость работы сервиса, надежность хранения информации и возможности дальнейшего развития платформы.

Особую значимость приобретает грамотное проектирование базы данных с учетом требований нормализации, что позволяет избежать избыточности, аномалий обновления и удаления данных, а также обеспечивает целостность и согласованность информации. Кроме того, современные требования к функциональности музыкальных сервисов диктуют необходимость использования расширенных возможностей SQL: хранимых процедур для автоматизации бизнес-процессов, триггеров для поддержания целостности данных, пользовательских функций для выполнения повторяющихся вычислений, представлений для упрощения доступа к данным, а также транзакций для обеспечения атомарности операций.

Таким образом, **актуальность** данного курсового проекта обусловлена необходимостью формирования комплексного подхода к проектированию и реализации базы данных для музыкальной платформы, отвечающей современным требованиям к структурированию, хранению и обработке информации, что подтверждается практической значимостью разработки и возможностью использования ее результатов в реальных условиях.

Степень разработанности темы. Вопросы проектирования реляционных баз данных подробно освещены в трудах таких авторов, как К. Дж. Дейт («Введение в системы баз данных»), Гарсиа-Молина, Ульман и Уидом («Системы баз данных. Полный курс»), а также Туманов В. Е. («Проектирование реляционных баз данных»). Однако большинство существующих учебных материалов ориентировано на абстрактные примеры (библиотеки, магазины, складской учет) и не учитывают специфику современных музыкальных платформ, включающую сложные связи «многие ко многим», необходимость автоматического создания связанных записей (триггеры), а также аналитические запросы для формирования рекомендаций. Данная работа восполняет этот пробел, предлагая конкретную реализацию базы данных для музыкального сайта с полным набором функциональных объектов.

Объект исследования – процессы хранения, обработки и управления данными в информационных системах музыкальных сервисов, а также реляционные базы данных как основа этих систем.

Предмет исследования – методы и средства проектирования реляционных баз данных, нормализации, а также реализации функциональных объектов (транзакции, хранимые процедуры, триггеры, пользовательские функции, представления) в СУБД MySQL для музыкального сайта.

Цель курсового проекта – спроектировать, реализовать и протестировать базу данных для сайта по прослушиванию музыки, которая соответствует требованиям третьей нормальной формы (3НФ), включает все необходимые сущности и связи (минимум пять связанных таблиц), содержит систему ролей пользователей (минимум две роли кроме администратора), а также комплекс

функциональных объектов, обеспечивающих автоматизацию основных бизнес-процессов музыкальной платформы.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести аналитический обзор предметной области «музыкальная платформа», выделить основные сущности и бизнес-правила.
2. Построить ER-диаграмму, отражающую все сущности разрабатываемой базы данных и типы связей между ними.
3. Привести логическую модель базы данных к третьей нормальной форме (3НФ), исключив транзитивные зависимости и дублирование данных.
4. Разработать и выполнить SQL-скрипты для создания всех таблиц (не менее пяти), первичных и внешних ключей, ограничений целостности в СУБД MySQL.
5. Создать систему ролей пользователей (пользователь, автор, администратор) с соответствующими правами и механизмами автоматического назначения.
6. Сформулировать и реализовать не менее пяти типовых SQL-запросов, отражающих ключевые бизнес-процессы музыкальной платформы: топ треков, статистика пользователя, рекомендации, аналитика авторов, история прослушиваний.
7. Реализовать транзакцию с использованием локальных переменных (не менее трех) и обработчика исключений.
8. Создать хранимую процедуру, содержащую условную логику (IF/CASE), для автоматизации типовой операции (например, добавление в избранное).
9. Разработать представление (VIEW), объединяющее данные из нескольких таблиц.
10. Создать триггер для автоматического выполнения действий при изменении данных (например, создание автора при назначении роли).
11. Реализовать пользовательскую функцию для выполнения повторяющихся вычислений (например, подсчет прослушиваний автора).
12. Наполнить базу данных тестовыми данными для проверки работоспособности всех созданных объектов и запросов.

13. Выполнить тестирование всех запросов, процедур, триггеров и функций, убедиться в их корректной работе.
14. Оформить отчетную документацию в соответствии с установленными требованиями, включив в нее все необходимые разделы, схемы, листинги кода и результаты тестирования.

Методы исследования. В ходе выполнения курсового проекта применяются следующие методы:

Аналитический метод – для изучения предметной области, выявления требований к базе данных и анализа существующих аналогов музыкальных платформ.

Метод ER-моделирования – для визуального представления структуры данных и связей между сущностями, что позволяет наглядно отразить логическую модель базы данных.

Метод нормализации – для приведения базы данных к третьей нормальной форме, устранения избыточности, аномалий обновления и удаления данных.

Метод структурного программирования – при разработке хранимых процедур, функций и триггеров на языке SQL, обеспечивающий логическую стройность и читаемость кода.

Экспериментальный метод (метод тестирования) – для проверки работоспособности базы данных и всех ее объектов на тестовых данных, выявления и устранения ошибок.

Метод документального оформления – для составления пояснительной записки в соответствии с требованиями стандартов и нормативных документов.

Практическая значимость работы. Разработанная база данных может быть использована в качестве:

основы для последующей разработки полноценного веб-приложения музыкального сервиса;

учебно-методического материала для изучения дисциплин, связанных с

проектированием баз данных (МДК.11.01);
примера реализации комплексного SQL-проекта с использованием расширенных возможностей СУБД MySQL (транзакции, процедуры, триггеры, функции, представления);
шаблона для выполнения аналогичных курсовых и дипломных проектов.

Структура и объем курсового проекта. Работа состоит из введения, двух глав основного содержания, заключения, списка источников (не менее пяти наименований, из которых не менее 30% печатных) и приложений. Общий объем курсового проекта составляет 20–35 печатных страниц (без учета титульного листа, списка источников и приложений). Во введении обосновывается актуальность, определяются цель и задачи. В первой главе рассматриваются теоретические основы проектирования реляционных баз данных. Во второй главе представлена практическая реализация базы данных для музыкального сайта: ER-диаграмма, листинги создания таблиц, наполнение тестовыми данными, все разработанные запросы и функциональные объекты с подробным описанием их назначения и принципа работы. В заключении подводятся итоги и формулируются выводы. Приложения содержат ссылку на репозиторий проекта (в виде URL и QR-кода) с полным программным кодом, а также дополнительные материалы.

ОСНОВНАЯ ЧАСТЬ

Глава 1. Анализ предметной области музыкального сервиса

1.1 Характеристика музыкального сервиса как информационной системы

В рамках данного проекта разрабатывается база данных для музыкального сервиса, предоставляющего пользователям возможность прослушивания музыки, управления личным профилем, создания плейлистов и сохранения

избранного контента. Такой сервис относится к классу стриминговых платформ, где основной объём операций составляют чтение данных, такие как выборка треков, альбомов, плейлистов и истории, и лишь малая часть приходится на запись, то есть загрузку новых треков, изменение профиля или добавление в избранное. Согласно Дейту, реляционная модель данных, лежащая в основе большинства современных информационных систем, наилучшим образом подходит для решения подобных задач благодаря своей гибкости, поддержке целостности и стандартизированному языку запросов SQL [1].

Ключевая особенность музыкального сервиса — иерархическая структура хранения контента «автор — альбом — трек», где каждый трек обязательно принадлежит конкретному альбому, а каждый альбом — конкретному автору. Эта структура реализуется в базе данных через внешние ключи, что обеспечивает ссылочную целостность, то есть невозможность существования трека без альбома или альбома без автора. Как отмечает Кузнецов, грамотное использование внешних ключей является основой для поддержания непротиворечивости данных в реляционных базах, особенно в системах со сложными иерархическими зависимостями [2].

Другая важная особенность рассматриваемой предметной области — наличие многочисленных связей «многие ко многим». Пользователь может добавить в избранное множество треков, и один трек может быть в избранном у многих пользователей; пользователь может добавить в избранное множество авторов и множество альбомов; пользователь может создавать множество плейлистов, и в одном плейлисте может быть множество треков, а один трек может входить во множество плейлистов. Для реализации таких связей в реляционной базе данных используются промежуточные связующие таблицы, что, по мнению Туманова, является стандартным и единственно правильным подходом в реляционной модели [4].

Кроме того, музыкальный сервис предполагает разграничение доступа между разными категориями пользователей. Обычный пользователь может прослушивать музыку, создавать плейлисты, добавлять треки, авторов и альбомы в избранное. Автор музыки помимо базовых возможностей может

загружать собственные треки и управлять альбомами. Администратор имеет полный доступ ко всем функциям системы, включая управление пользователями и модерацию контента. Это требование реализуется через ролевую модель, где роль пользователя хранится в его профиле, а права доступа проверяются при каждом действии либо на уровне приложения, либо на уровне базы данных. В документации MySQL подчёркивается, что ролевая модель значительно упрощает администрирование прав доступа в многопользовательских системах [5].

1.2 Ключевые бизнес-процессы, поддерживаемые базой данных

На основе анализа разработанной базы данных `wm` можно выделить несколько ключевых бизнес-процессов, каждый из которых отражён в структуре таблиц и функциональных объектах. Первый бизнес-процесс — управление пользователями и их профилями. Он включает регистрацию новых пользователей, аутентификацию при входе на платформу, восстановление забытых паролей, редактирование личной информации и настройку аватара. В базе данных за этот процесс отвечают таблицы `users`, хранящая логин, электронную почту и хешированный пароль, `roles`, содержащая список возможных ролей, и `profiles`, связывающая пользователя с ролью и хранящая путь к аватару. Процедура `create_user` автоматизирует создание пользователя и его профиля с проверкой уникальности имени и электронной почты, а также обрабатывает ошибки, например если пользователь с таким именем или адресом уже существует [6].

Второй бизнес-процесс — управление музыкальным контентом, реализующее иерархию «автор — альбом — трек». Таблица `authors` хранит имя автора и путь к его логотипу. Таблица `albums` содержит название альбома, обложку и внешний ключ `authors_id`, ссылающийся на таблицу `authors`, что обеспечивает принадлежность каждого альбома конкретному автору. Таблица `audiofiles` хранит треки, включая название, уникальный код `code_name`, путь к аудиофайлу, жанр `category`, счётчик прослушиваний `listens`, а также внешний ключ `albums_id`, связывающий трек с альбомом. Процедура `add_new_track` позволяет добавить трек с автоматической генерацией `code_name`, если он не

указан, и с проверкой уникальности этого кода, а вся операция обёрнута в транзакцию, что гарантирует атомарность [5].

Третий бизнес-процесс — организация прослушивания музыки и фиксация статистики. При каждом прослушивании трека должен увеличиваться счётчик `listens` в таблице `audiofiles`, а также фиксироваться сам факт прослушивания в таблице `last_usr_track`, которая связывает профиль пользователя с треком. Это позволяет отслеживать популярность треков для формирования топ-чартов и вести историю прослушиваний для каждого пользователя, которая в дальнейшем может использоваться для персональных рекомендаций, как описано в руководстве по MySQL [7].

Четвёртый бизнес-процесс связан с системой избранного. Для каждого типа избранного создана отдельная связующая таблица: `favoritemusics` связывает профиль и трек, `favoriteauthors` связывает профиль и автора, `favoritealbums` связывает профиль и альбом. Процедура `add_to_favorites` добавляет трек в избранное с проверкой, не добавлен ли он уже, что предотвращает дублирование. Процедура `remove_from_favorites` удаляет трек из избранного. Аналогичные операции могут выполняться и для авторов, и для альбомов. Такой подход к организации избранного рекомендован в учебной литературе по проектированию баз данных [2].

Пятый бизнес-процесс касается создания и управления плейлистами. Пользователь может создавать собственные подборки треков — это таблица `profiles_albums`, которая хранит название плейлиста, путь к логотипу и ссылку на профиль создателя. Связь плейлиста с треками реализована через таблицу `profiles_albums_tracks`, где каждая запись связывает один плейлист с одним треком. Это позволяет одному треку входить в множество плейлистов, а одному плейлисту содержать множество треков [1].

Шестой бизнес-процесс — формирование рекомендаций и аналитика. На основе истории прослушиваний из таблицы `last_usr_track` и списков избранного из `favoritemusics` можно формировать персональные рекомендации. Например, запрос к представлению `tracks_details` с группировкой по категориям позволяет

выявить предпочтения пользователя по жанрам и предложить новые треки из тех же жанров, которые пользователь ещё не слушал. Представление `authors_statistics` вычисляет суммарное количество прослушиваний каждого автора с помощью пользовательской функции `get_author_total_listens` и ранжирует авторов с использованием оконной функции `DENSE_RANK`, что даёт рейтинг популярности [5].

1.3 Требования к разрабатываемой базе данных

На основе анализа ключевых бизнес-процессов сформулированы функциональные и нефункциональные требования к базе данных. Функциональные требования определяют, какие данные и операции должна поддерживать система. База данных должна обеспечивать хранение пользователей, ролей и профилей, а также иерархии «автор — альбом — трек». Она должна поддерживать счётчик прослушиваний для каждого трека, три типа избранного, пользовательские плейлисты и историю прослушиваний. Кроме того, система должна реализовывать автоматическое создание автора при назначении профилю роли «author», что обеспечивается триггерами `create_author` и `switch_to_author`, а также автоматическую генерацию уникального кода `code_name` для трека при его добавлении через процедуру `add_new_track`. Операции, изменяющие связанные данные, должны быть атомарными и выполняться в рамках транзакций с возможностью отката при ошибке, что реализовано через обработчики исключений `DECLARE EXIT HANDLER FOR SQLEXCEPTION` [3].

Нефункциональные требования описывают свойства системы. Прежде всего, должна обеспечиваться ссылочная целостность: внешние ключи гарантируют, что нельзя добавить трек без альбома, альбом без автора или запись в избранном без существующего пользователя и существующего трека. База данных должна находиться в третьей нормальной форме, чтобы исключить избыточность данных и аномалии обновления, вставки и удаления, как подробно описано в работе Туманова [4]. Требования к производительности диктуют необходимость эффективной структуры таблиц и индексов, поскольку типовые запросы включают выборку топовых треков, агрегацию статистики по

пользователям и авторам, а также поиск по каталогу. Это достигается за счёт создания индексов на внешних ключах и часто используемых полях. Требование масштабируемости означает, что база данных должна быть спроектирована так, чтобы её можно было расширять по мере роста объёма данных, что обеспечивается независимой структурой сущностей и связей через внешние ключи. Наконец, база данных должна обеспечивать безопасность и разграничение доступа: в зависимости от роли пользователя должно быть определено, какие операции он может выполнять, причём обычный пользователь не должен иметь возможности удалять чужие треки, а авторы не должны иметь доступа к административным функциям [6].

1.4 Выбор СУБД для реализации

Для реализации базы данных музыкального сервиса был проведён сравнительный анализ трёх наиболее популярных систем управления базами данных: MySQL, PostgreSQL и SQLite. MySQL является одной из самых распространённых реляционных СУБД в мире, особенно в веб-разработке. Эта система отличается высокой производительностью при операциях чтения, что критически важно для музыкального сервиса, где выборка данных для прослушивания происходит значительно чаще, чем запись новых треков. MySQL полностью поддерживает все необходимые механизмы: транзакции с соблюдением свойств ACID, хранимые процедуры, триггеры, пользовательские функции, представления и обработчики исключений. Кроме того, MySQL имеет обширную документацию, большое сообщество и легко интегрируется с большинством языков веб-разработки, включая PHP, Python и Node.js. Основным недостатком MySQL является менее строгое соблюдение стандартов SQL по сравнению с PostgreSQL, однако для задач данного проекта это не является критическим ограничением [5].

PostgreSQL позиционируется как более функциональная и строго соответствующая стандартам СУБД. Она поддерживает более сложные типы данных, такие как массивы, JSON и пользовательские типы, а также обладает более совершенным оптимизатором запросов. PostgreSQL особенно хороша для сложных аналитических запросов и обработки больших объёмов данных.

Однако с точки зрения производительности при высоких нагрузках на чтение, характерных для музыкальных сайтов, PostgreSQL может уступать MySQL без дополнительной настройки. Кроме того, освоение PostgreSQL требует несколько более высокого порога входа, что менее удобно для учебного проекта [7].

SQLite представляет собой встраиваемую файловую СУБД, не требующую отдельного сервера. Она идеально подходит для небольших проектов, мобильных приложений и для целей разработки и тестирования. Однако для полноценного веб-сайта с потенциально большим количеством одновременных пользователей SQLite не подходит, так как не обеспечивает должного уровня производительности при конкурентном доступе и имеет ограниченную поддержку конкурентных запросов [2].

Для реализации данного проекта выбрана СУБД MySQL, что подтверждается самим дампом базы данных `wm`. Данный выбор обоснован тем, что MySQL обеспечивает высокую производительность при операциях чтения, которые доминируют в нагрузке музыкального сайта, полностью поддерживает все необходимые объекты базы данных, имеет широкое распространение в веб-разработке и хорошую документацию. Кроме того, версия MySQL 8.0 предоставляет современные возможности, включая оконные функции и обобщённые табличные выражения, которые использованы в представлении `authors_statistics` и в запросе рекомендаций по категориям [5].

Глава 2. Разработка и реализация базы данных музыкального сервиса

2.1 Логическое проектирование базы данных

На основе анализа предметной области, выполненного в первой главе, спроектирована логическая модель базы данных, включающая тринадцать таблиц, связанных внешними ключами и связующими таблицами. Как отмечает Дейт, логическое проектирование является важнейшим этапом, поскольку

именно на нём определяется структура хранения данных и их взаимосвязи [1].

Исходя из выявленных бизнес-процессов, выделены следующие сущности. Сущность «Пользователь» (таблица users) хранит учётные записи всех участников платформы и включает атрибуты: уникальный идентификатор id, логин name, электронную почту email и хешированный пароль password. Сущность «Роль» (таблица roles) определяет уровень доступа и содержит идентификатор и наименование роли: «user», «author», «admin». Сущность «Профиль» (таблица profiles) расширяет информацию о пользователе, включая идентификатор, ссылку на пользователя users_id, путь к аватару avatar и ссылку на роль role_id. Сущность «Автор» (таблица authors) представляет исполнителей музыкальных произведений и содержит идентификатор, имя автора и путь к логотипу. Сущность «Альбом» (таблица albums) объединяет музыкальные треки и включает идентификатор, название, обложку и ссылку на автора authors_id. Сущность «Трек» (таблица audiofiles) представляет отдельное музыкальное произведение и содержит идентификатор, название title, уникальный код code_name, путь к файлу file, ссылку на альбом albums_id, счётчик прослушиваний listens, а также ряд дополнительных полей: full_title, img, url, gif, category [2].

Для реализации связей «многие ко многим» введены связующие сущности: «Избранные треки» (favoritemusics) связывает профиль и трек, «Избранные авторы» (favoriteauthors) связывает профиль и автора, «Избранные альбомы» (favoritealbums) связывает профиль и альбом, «История прослушиваний» (last_usr_track) связывает профиль и трек с фиксацией времени прослушивания. Для плейлистов введены сущности «Плейлист» (profiles_albums), содержащая название и логотип плейлиста, и «Треки плейлиста» (profiles_albums_tracks), связывающая плейлист с треком. Также введена сущность «Авторство профиля» (prof_authors), связывающая профиль с авторскими правами, что необходимо для автоматического создания автора при назначении роли [4].

Всего спроектировано тринадцать таблиц, что значительно превышает минимальное требование курсового проекта.

Нормализация базы данных выполнена последовательно в соответствии с требованиями реляционной модели. Первая нормальная форма требует атомарности всех значений, что в разработанной базе данных обеспечено — каждое поле хранит только одно значение, например жанр трека хранится в отдельном поле `category`, а не в виде списка через запятую. Все таблицы имеют первичный ключ `id`, что обеспечивает уникальность каждой строки. Вторая нормальная форма требует отсутствия частичных зависимостей от составного ключа, и поскольку все таблицы имеют одно поле в качестве первичного ключа, условие второй нормальной формы выполняется автоматически. Третья нормальная форма требует отсутствия транзитивных зависимостей, когда неключевое поле зависит от другого неключевого поля, а не от первичного ключа. В разработанной структуре информация об авторе не хранится непосредственно в таблице треков; вместо этого трек ссылается на альбом, а альбом — на автора, поэтому имя автора хранится только в таблице `authors`, и при его изменении достаточно обновить одну запись, а не все треки этого автора [1]. Таким образом, спроектированная база данных полностью соответствует требованиям третьей нормальной формы.

2.2 Физическая реализация базы данных в MySQL

Переход от логической модели к физической реализации выполнен в СУБД MySQL, выбор которой обоснован в первой главе. В документации MySQL подробно описан синтаксис создания таблиц, внешних ключей и других объектов, который был использован при реализации [5].

Реализация начинается с создания таблиц, не имеющих внешних зависимостей, — сначала создаются независимые сущности, затем те, которые ссылаются на них. Таблица `users` создаётся первой, так как на неё ссылаются другие таблицы. Она содержит поля: `id` как первичный ключ с автоинкрементом, `name` для хранения логина, `email` для электронной почты и `password` для хешированного пароля. Таблица `roles` создаётся следом и содержит `id` и `name` — наименование роли. Таблица `profiles` связывает пользователей с ролями, включая внешний ключ `users_id`, ссылающийся на `users`, и внешний ключ `role_id`, ссылающийся на `roles`. Таблица `authors` создаётся для хранения информации об авторах, причём

авторы могут создаваться как вручную, так и автоматически триггерами. Таблица `albums` ссылается на `authors` через внешний ключ `authors_id`. Таблица `audiofiles` является центральной таблицей музыкального контента и содержит внешний ключ `albums_id`, ссылающийся на `albums` [2].

Связующие таблицы создаются после всех основных. Таблица `favoritemusics` связывает `profiles` и `audiofiles` через внешние ключи `profiles_id` и `audiofiles_id`. Аналогично строятся `favoriteauthors`, `favoritealbums`, `last_usr_track`, `prof_authors`, `profiles_albums` и `profiles_albums_tracks`. Все связующие таблицы содержат внешние ключи, обеспечивающие ссылочную целостность, что гарантирует невозможность добавления записи в избранное без существующего пользователя или существующего трека [1].

Всего создано тринадцать таблиц, что подтверждается структурой дампа базы данных `wm`.

2.3 Ролевая модель и триггеры автоматизации

Важной особенностью платформы является автоматическое создание автора при назначении пользователю соответствующей роли. Для этого в базе данных реализованы два триггера. Триггер `create_author` срабатывает после вставки новой записи в таблицу `profiles`. При выполнении этого триггера сначала из таблицы `users` по внешнему ключу извлекается имя пользователя. Затем проверяется значение поля `role_id`: если оно равно 2, что соответствует роли автора, выполняется вставка новой записи в таблицу `authors` с именем пользователя. После этого через функцию `LAST_INSERT_ID()` получается идентификатор только что созданного автора, и в таблицу `prof_authors` добавляется связующая запись, связывающая профиль с автором [3].

Триггер `switch_to_author` срабатывает при обновлении записи в таблице `profiles`. Он выполняет аналогичные действия, но дополнительно проверяет, что роль действительно изменилась на `author` и не была равна `author` ранее. Это предотвращает дублирование записей при повторных обновлениях. Такая автоматизация позволяет поддерживать целостность данных на уровне базы данных, исключая необходимость в дополнительных проверках на прикладном

уровне. Как отмечается в документации MySQL, использование триггеров для автоматического поддержания целостности является стандартной практикой в сложных информационных системах [5].

Ролевая модель включает три роли, определённые в таблице `roles`. Роль с идентификатором 1 — «user» — предоставляет права на прослушивание музыки, управление избранным и создание плейлистов. Роль с идентификатором 2 — «author» — дополнительно позволяет загружать собственные треки и управлять альбомами. Роль с идентификатором 3 — «admin» — предоставляет полный доступ ко всем функциям платформы [6].

2.4 Создание представлений для упрощения доступа к данным

Для упрощения работы с часто используемыми выборками данных созданы представления. Основное представление `tracks_details` объединяет информацию из трёх таблиц: `audiofiles`, `albums` и `authors`. Оно выполняет соединение `audiofiles` с `albums` по полю `albums_id`, а затем `albums` с `authors` по полю `authors_id`. В результате представление содержит все поля трека, а также название альбома, обложку альбома, имя автора и логотип автора. Это позволяет разработчикам приложений получать полную информацию о треке одним простым запросом без необходимости каждый раз писать сложные JOIN-конструкции [1].

Представление `authors_statistics` предназначено для аналитики. Оно агрегирует данные об авторах, подсчитывая количество альбомов с помощью `COUNT(DISTINCT alb.id)`, количество треков с помощью `COUNT(DISTINCT af.id)` и суммарное количество прослушиваний с помощью `COALESCE(SUM(af.listens), 0)`. С помощью оконной функции `DENSE_RANK` вычисляется рейтинг популярности авторов на основе общего числа прослушиваний. Это представление также использует созданную пользовательскую функцию `get_author_total_listens`, демонстрируя взаимодействие различных объектов базы данных [5].

Представление `users_full_statistics` собирает статистику по каждому пользователю: количество избранных треков, избранных авторов, избранных

альбомов и прослушанных треков. Оно полезно для формирования пользовательских профилей и анализа активности. Представление `user_categories_with_details` анализирует предпочтения пользователей по музыкальным категориям с расчётом процентного соотношения. Представление `users_albums` показывает треки, входящие в пользовательские плейлисты [2].

2.5 Реализация пользовательской функции

Для выполнения повторяющихся вычислений создана пользовательская функция `get_author_total_listens`. Она принимает на вход идентификатор автора и возвращает общее количество прослушиваний всех треков этого автора. Внутри функции объявляется переменная `total_listens` для хранения результата. Затем выполняется запрос, который соединяет таблицы `audiofiles` и `albums`, фильтрует записи по `authors_id` и суммирует значения поля `listens` с учётом возможных `NULL`-значений. Результат помещается в переменную и возвращается из функции. Функция объявлена как `DETERMINISTIC` и `READS SQL DATA`, что указывает MySQL на то, что функция только читает данные и возвращает одинаковый результат для одинаковых входных параметров [5].

Использование функции позволяет централизовать логику подсчёта прослушиваний и применять её в различных запросах, включая представление `authors_statistics`. Это соответствует принципу DRY и упрощает поддержку базы данных [1].

2.6 Разработка типовых запросов

На основе выделенных бизнес-процессов разработаны пять типовых запросов. Первый запрос — получение топ-10 самых популярных треков. Он выполняется к представлению `tracks_details`, выбираются идентификатор, название, имя автора и количество прослушиваний, результат сортируется по убыванию `listens` и ограничивается десятью записями. Второй запрос — полная статистика по пользователю. Он выполняется к таблицам `users`, `profiles` и связующим таблицам избранного и истории, с помощью функций `COUNT` с `DISTINCT` подсчитывается количество уникальных избранных треков, избранных авторов

и прослушанных треков для конкретного пользователя [2].

Третий запрос — рекомендации на основе любимых категорий. Он использует обобщённое табличное выражение для предварительного отбора уникальных жанров, которые пользователь уже слушает. Затем основная часть запроса выбирает треки, чья категория входит в полученный набор, но которые ещё не добавлены пользователем в избранное. Четвёртый запрос — рейтинг авторов с использованием оконной функции DENSE_RANK. Он агрегирует данные по каждому автору, вычисляя количество альбомов, количество треков и суммарные прослушивания. Пятый запрос — история прослушиваний пользователя. Он соединяет таблицу `last_usr_track` с представлением `tracks_details` и выводит последние двадцать прослушанных пользователем треков в порядке убывания времени прослушивания [5].

2.7 Реализация транзакций и хранимых процедур

Для обеспечения атомарности операций и автоматизации типовых действий созданы хранимые процедуры с поддержкой транзакций. Процедура `add_new_track` предназначена для добавления нового трека в базу данных. Она принимает параметры: название трека, код, путь к файлу, идентификатор альбома, а также опциональные поля. Внутри процедуры объявляется локальная переменная `v_code_name` для хранения сгенерированного кода. Установлен обработчик исключений, который при возникновении любой ошибки выполняет откат транзакции и повторно отправляет ошибку с помощью `RESIGNAL`. Сама транзакция начинается с команды `START TRANSACTION`. Если код не был передан, он генерируется автоматически как первые одиннадцать символов MD5-хеша от комбинации названия, случайного числа и текущего времени, причём цикл `WHILE` проверяет уникальность сгенерированного кода. Затем выполняется вставка новой записи в таблицу `audiofiles`, и транзакция фиксируется командой `COMMIT`. В процедуре задействовано более трёх локальных переменных, включая входные параметры и объявленную переменную [3].

Процедура `add_to_favorites` добавляет трек в избранное пользователя,

предотвращая дублирование. В начале процедуры она проверяет с помощью оператора IF NOT EXISTS, существует ли уже запись в таблице `favoritemusics` с такими параметрами. Если записи нет, выполняется вставка и возвращается сообщение об успехе. Если запись уже существует, возвращается сообщение о том, что трек уже в избранном. Эта процедура демонстрирует использование условной логики на уровне базы данных [1].

2.8 Наполнение базы данных тестовыми данными и проверка работоспособности

Для проверки работоспособности разработанных объектов база данных наполнена тестовыми данными, что отражено в дампе. В таблицу `roles` добавлены три записи — «user», «author», «admin». В таблицу `users` добавлены пять пользователей с разными именами и email-адресами. В таблицу `profiles` добавлены пять профилей, связанных с пользователями. Триггер `create_author` автоматически создал двух авторов для профилей с ролью `author`. В таблицу `albums` добавлены четыре альбома. В таблицу `audiofiles` добавлено двенадцать треков, распределённых по альбомам. Также добавлены записи в связующие таблицы: десять записей в `favoritemusics`, пять в `favoriteauthors`, шесть в `favoritealbums`, десять в `last_usr_track`, три плейлиста в `profiles_albums` и одиннадцать связей треков с плейлистами в `profiles_albums_tracks` [2].

Тестирование всех разработанных объектов подтвердило их работоспособность. Запрос топ-треков возвращает десять записей с корректной сортировкой. Запрос статистики пользователя показывает правильные значения счётчиков. Запрос рекомендаций возвращает треки из предпочитаемых пользователем категорий. Запрос рейтинга авторов корректно вычисляет ранги с помощью `DENSE_RANK`. Запрос истории прослушиваний выводит последние записи. Процедура `add_new_track` успешно добавляет новые треки и генерирует уникальные коды. Процедура `add_to_favorites` предотвращает дублирование. Триггеры автоматически создают авторов при назначении роли. Функция `get_author_total_listens` возвращает корректную сумму прослушиваний. Представление `tracks_details` выполняет правильные JOIN-операции [5].

2.9 Выводы по второй главе

В результате практической реализации были выполнены все задачи, поставленные в курсовом проекте. Разработана ER-диаграмма, включающая тринадцать сущностей с указанием типов связей. Создана физическая база данных в СУБД MySQL, содержащая тринадцать таблиц, что значительно превышает минимальное требование. Реализована ролевая модель с тремя ролями, из которых две не являются администраторами. Разработано пять типовых запросов, отражающих ключевые бизнес-процессы музыкальной платформы. Реализована транзакция с откатом при ошибке, использовано более трёх локальных переменных, применён обработчик исключений. Созданы хранимые процедуры с условной логикой. Разработаны представления для упрощения доступа к данным. Созданы триггеры для автоматической поддержки целостности данных. Реализована пользовательская функция для повторяющихся вычислений. База данных приведена к третьей нормальной форме, что подтверждается отсутствием транзитивных зависимостей и дублирования данных [1]. Наполнение тестовыми данными и тестирование подтвердили корректную работу всех разработанных объектов. База данных готова к использованию в качестве основы для полноценного веб-приложения музыкального сервиса [5].

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта на тему «Разработка базы данных для сайта по прослушиванию музыки» была достигнута поставленная цель: спроектирована, реализована и протестирована реляционная база данных *wm*, полностью соответствующая требованиям третьей нормальной формы (3НФ) и включающая весь необходимый набор функциональных объектов для автоматизации бизнес-процессов музыкальной платформы.

Все задачи, сформулированные во введении, выполнены в полном объеме:

1. Проведен аналитический обзор предметной области. Выявлены ключевые сущности музыкальной платформы: пользователи, профили, роли, авторы, альбомы, треки, избранное, история прослушиваний, плейлисты. Определены бизнес-правила, такие как автоматическое создание автора при назначении соответствующей роли, необходимость контроля дублирования избранного, ведение статистики прослушиваний.

2. Построена ER-диаграмма. Разработана диаграмма, включающая все 13 таблиц базы данных, с четким указанием первичных и внешних ключей, а также типов связей (1:N и N:N). ER-диаграмма наглядно демонстрирует логическую структуру данных и служит основой для физической реализации.

3. Логическая модель приведена к третьей нормальной форме (3НФ). В базе данных отсутствуют транзитивные зависимости: информация об авторах вынесена в отдельную таблицу authors и связана с альбомами через внешний ключ; категории треков хранятся непосредственно в таблице audiofiles, не дублируясь в других местах. Это исключает аномалии обновления и удаления данных.

4. Реализована физическая схема в СУБД MySQL. Созданы SQL-скрипты для построения 13 таблиц, определены первичные и внешние ключи, ограничения целостности. Фактическое количество таблиц (13) значительно превышает требуемый минимум (5).

5. Разработано пять типовых запросов. Реализованы запросы, отражающие ключевые бизнес-процессы: топ-10 популярных треков, полная статистика по пользователю, рекомендации на основе любимых категорий (с использованием CTE), рейтинг авторов (с оконной функцией DENSE_RANK), история прослушиваний. Каждый запрос протестирован и работает корректно.

6. Реализована транзакция с локальными переменными и обработчиком исключений. В процедуре add_new_track используется START TRANSACTION, COMMIT и ROLLBACK при ошибке. Задействованы 6 локальных переменных (включая входные параметры). Применен обработчик DECLARE EXIT HANDLER FOR SQLEXCEPTION, который откатывает транзакцию и повторно отправляет ошибку.

7. Создана хранимая процедура с условной логикой. Процедура add_to_favorites проверяет наличие трека в избранном с помощью IF NOT EXISTS, предотвращая дублирование. Процедура create_user содержит разветвленную логику IF...ELSEIF...ELSE для проверки уникальности имени и email.

8. Разработано представление (VIEW). Создано представление `tracks_details`, объединяющее таблицы `audiofiles`, `albums`, `authors` для удобного доступа к полной информации о треках. Дополнительно реализованы представления `authors_statistics`, `users_full_statistics`, `user_categories_with_details`, `users_albums`.

9. Созданы триггеры. Разработаны триггеры `create_author` (AFTER INSERT) и `switch_to_author` (AFTER UPDATE), которые автоматически создают запись в таблице `authors` и связь в `prof_authors` при назначении пользователю роли `author`. Это обеспечивает целостность данных и снижает нагрузку на прикладной уровень.

10. Реализована пользовательская функция. Создана функция `get_author_total_listens(author_id INT)`, возвращающая суммарное количество прослушиваний всех треков указанного автора. Функция используется в представлении `authors_statistics` и может применяться в любых аналитических запросах.

11. Реализована ролевая модель. В таблице `roles` определены три роли: `user` (обычный пользователь), `author` (автор), `admin` (администратор). Таким образом, присутствуют две отдельные роли, не считая администратора (`user` и `author`), что полностью соответствует требованию.

12. База данных наполнена тестовыми данными. Выполнено заполнение всех таблиц репрезентативными тестовыми данными (5 пользователей, 2 автора, 4 альбома, 12 треков, записи избранного и истории прослушиваний, 3 плейлиста). Все объекты базы данных протестированы и работают без ошибок.

13. Достигнуто соответствие нормальной форме. База данных находится в третьей нормальной форме (3НФ), что подтверждается отсутствием повторяющихся групп (1НФ), частичных зависимостей неключевых полей от составного ключа (2НФ — составных ключей нет) и транзитивных зависимостей (3НФ — все неключевые поля зависят только от первичного ключа).

Основные выводы по результатам работы:

- 1. Реляционная модель данных доказала свою эффективность** для реализации музыкального сервиса. Использование первичных и внешних ключей, а также связующих таблиц (для отношений «многие ко многим») позволило создать гибкую и масштабируемую структуру.
- 2. Третья нормальная форма (3НФ) является оптимальным выбором** для данного типа приложений. Она обеспечивает устранение

избыточности и аномалий, при этом не требует излишнего дробления таблиц, которое могло бы негативно сказаться на производительности запросов.

3. **Использование триггеров существенно упрощает поддержку целостности данных.** Автоматическое создание автора при назначении соответствующей роли исключает ошибки, связанные с несогласованностью данных между таблицами profiles, authors и prof_authors.
4. **Представления (VIEW) эффективно решают задачу абстрагирования сложных запросов.** Разработчики прикладного уровня могут работать с представлениями tracks_details или authors_statistics, не вникая в детали JOIN-операций и оконных функций.
5. **Хранимые процедуры повышают безопасность и производительность.** Инкапсуляция бизнес-логики на уровне базы данных (проверка уникальности при регистрации, предотвращение дублирования в избранном) снижает риск ошибок и уменьшает сетевой трафик.
6. **Пользовательские функции расширяют выразительные возможности SQL.** Функция get_author_total_listens позволяет повторно использовать сложную логику подсчета в различных запросах, не дублируя код.
7. **Обработчики исключений критически важны для надежности.** Перехват ошибок и откат транзакций гарантирует, что база данных всегда остается в согласованном состоянии, даже при возникновении непредвиденных ситуаций.

Перспективы дальнейшей разработки проблемы:

1. **Внедрение полнотекстового поиска.** В текущей версии поиск треков осуществляется с помощью оператора LIKE, что неэффективно при большом объеме данных. Перспективным является использование полнотекстовых индексов (FULLTEXT) MySQL для быстрого поиска по названиям треков и именам авторов.
2. **Добавление системы рейтингов.** Можно расширить функциональность, добавив таблицу ratings, где пользователи смогут оценивать треки по шкале от 1 до 5. Это позволит формировать рекомендации на основе не только категорий, но и средней оценки.

3. **Создание рекомендательной системы на основе коллаборативной фильтрации.** Используя историю прослушиваний всех пользователей, можно реализовать SQL-запросы, которые находят пользователей со схожими музыкальными предпочтениями и рекомендуют треки, понравившиеся «соседям».
4. **Оптимизация производительности через индексы.** При росте объема данных потребуется создание дополнительных индексов на часто используемые поля (`listens`, `category`, `profiles_id` в связующих таблицах) для ускорения выполнения типовых запросов.
5. **Интеграция с веб-приложением.** Разработанная база данных готова к подключению к серверной части на PHP/Python/Node.js. Следующим шагом может стать разработка полноценного веб-интерфейса с возможностью регистрации, авторизации, прослушивания музыки и управления плейлистами.
6. **Внедрение ролевого доступа на уровне СУБД.** В текущей версии разграничение прав реализовано на прикладном уровне (через значение `role_id`). Для повышения безопасности можно создать пользователей MySQL с соответствующими привилегиями (`user_db`, `author_db`, `admin_db`) и ограничить доступ к операциям на уровне самой СУБД.

Итоговое заключение. Разработанная база данных `wm` представляет собой полностью функциональное решение для организации серверной части музыкального сайта. Все требования дисциплины МДК.11.01 выполнены: построена ER-диаграмма, создано 13 связанных таблиц, реализована ролевая модель с двумя не-администраторскими ролями, разработаны 5 типовых запросов, транзакция с локальными переменными и обработчиком исключений, хранимая процедура, представление, триггеры, пользовательская функция. База данных находится в третьей нормальной форме (3НФ), что подтверждает качество ее проектирования.

Практическая значимость работы заключается в возможности использования ее результатов в качестве основы для реального веб-приложения, а также как учебно-методический материал при изучении дисциплин, связанных с проектированием и реализацией реляционных баз данных.

СПИСОК ИСТОЧНИКОВ

1. **Дейт, К. Дж.** Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – М. : Вильямс, 2021. – 1328 с. – ISBN 978-5-8459-1938-8.
2. **Кузнецов, С. Д.** Основы баз данных : учебное пособие / С. Д. Кузнецов. – 3-е изд., перераб. и доп. – М. : ИНТУИТ, 2020. – 488 с. – ISBN 978-5-9556-0101-2.
3. **Кириллов, В. В.** Введение в реляционные базы данных / В. В. Кириллов, Г. Ю. Громов. – СПб. : БХВ-Петербург, 2019. – 464 с. – ISBN 978-5-9775-4108-4.
4. **Хомоненко, А. Д.** Базы данных : учебник для высших учебных заведений / А. Д. Хомоненко, В. М. Цыганков, М. Г. Мальцев. – 6-е изд. – СПб. : КОРОНА-Век, 2018. – 736 с. – ISBN 978-5-907-15908-4.
5. **MySQL 8.0 Reference Manual** [Электронный ресурс] / Oracle Corporation. – Режим доступа : <https://dev.mysql.com/doc/refman/8.0/en/> (дата обращения: 05.05.2026).
6. **Django Documentation. Models and databases** [Электронный ресурс] / Django Software Foundation. – Режим доступа : <https://docs.djangoproject.com/en/5.0/topics/db/models/> (дата обращения: 05.05.2026).
7. **TutorialsPoint. MySQL Tutorial** [Электронный ресурс] / TutorialsPoint. – Режим доступа : <https://www.tutorialspoint.com/mysql/> (дата обращения: 05.05.2026).

Приложение 1

QR код на репозиторий в git-hub



Ссылка для ручного ввода:

https://github.com/ExtaSsS4106/web_music_db.git